

Ephemerality Is a Transition Property: Obligation-Unsound Reconciliation in Kubernetes and Other Control Loops

James Beck Independent Researcher

Date: 2026-05-14

Series: Δ t Framework, Paper 27

Status: v0.2 / 1.0-candidate (2026-05-14).

Abstract

Reconciliation controllers are commonly specified in terms of convergence: given a desired state, the controller should eventually drive observed state toward that target and maintain it. This paper identifies a complementary safety property missing from that correctness vocabulary: *transition admissibility*. A reconciliation transition may restore desired state while silently degrading the obligations required for later diagnosis, governance, or operator reliance — including evidence, causality, custody, authority, continuity, and substrate accountability. We model Kubernetes-style controllers as operating over lossy projections of substrate state. When the projection collapses substrate states that carry different operator obligations, the controller produces an *obligation-unsound quotient*: convergence-correct, operator-unsound. We define ephemerality not as an object property but as a transition property under declared obligations and retention horizons. Three audit states — *Masked recovery*, *Orphaned causality*, *DegradedWithReceipt* — distinguish failure modes from constructive use of an open finding. A receipt durability invariant requires receipts to outlive the longest-lived affected obligation. Three worked Kubernetes scenarios exercise the vocabulary and force the audit-state distinctions. The result is a formal complement to convergence-oriented controller verification [4]: a controller may be correct with respect to desired-state convergence while remaining unsound with respect to operational testimony. P27 sits at the constructive endpoint of the P22–P26 negative spine: P25 [1] establishes that substitution is structurally forced under observability asymmetry; P26 [2] formalizes empty-window failure under temporal detachment; P27 names how to govern transitions so that what was sensed is not silently destroyed.

1. Introduction

A reconciliation controller observes a control-plane projection of a system, compares it against a declared desired state, and executes transitions that move the projection toward the target. The canonical correctness specification is *eventual convergence*: under reasonable assumptions, current state reaches and remains at desired state. Recent formal-methods work has produced machine-checked liveness proofs for production Kubernetes controllers operating under this specification [4]. Convergence is real, useful, and verifiable.

It is also incomplete. Consider a routine reconciliation: a pod fails on a node with degrading disk hardware; the controller drains the pod, destroying the container’s logs and the kernel disk-error messages tied to the pod’s lifecycle; a replacement is scheduled, possibly onto another node in the same node class. From the control plane’s perspective, the loop converged: the replica count is correct, the pod is Ready, the service is available. From the operator’s perspective, the evidence that would have linked the failure to a

substrate-disk problem was destroyed by the action that restored convergence. The next pod scheduled to that node class inherits the same risk; no record of the substrate accusation survives.

This is not a convergence failure. It is a *testimony* failure. The control plane optimized for the property it was specified to satisfy, and in doing so silently degraded a property no specification asked about: whether what was sensed survives the transition that restored the system.

Three results in this series describe complementary failure shapes for controllers operating on partial information. *Epistemic Border Control* (Paper 25 [1]) names *spatial substitution*: when a target variable lies outside the observability subspace, the controller is structurally forced to regulate a proxy in its place; the controller cannot recover what it cannot sense. *Premature Commitment, Belated Consequence* (Paper 26 [2]) names *temporal detachment*: when the time-axis evidence required for legitimate binding does not arrive within the consequence-viable window, no time supports both legitimacy and effect; the controller cannot bind what cannot mature in time. This paper names *transition admissibility*: even when the controller cannot bind correctly under prior constraints, it can be required not to silently destroy what was sensed during the actions it does take. P25 limits what can be regulated; P26 limits when binding can attach; P27 governs what must survive the transitions in between.

The paper makes four contributions:

1. **A formal model** of reconciliation controllers as quotient machines operating over lossy projections of substrate state (§3). Where a substrate B admits a projection $\alpha : B \rightarrow K$ into the control-plane visible state, two substrate states b_1, b_2 with distinct operator obligations may share $\alpha(b_1) = \alpha(b_2)$ — they are *quotient-equivalent* under α . A controller that operates over K produces an obligation-unsound quotient whenever it conflates such states.
2. **Transition admissibility** as a safety property complementary to convergence (§3). A reconciliation transition τ is admissible iff, for every obligation ω affected by τ , the transition produces a non-unaccounted Accounting outcome and the receipt horizon outlives ω 's horizon.
3. **An audit-state vocabulary** (§4) for naming where a transition lands: *Observed, Confirmed, Superseded, Masked, Orphaned, DegradedWithReceipt, Resolved, Invalidated, CannotTestify*. *Masked* and *Orphaned* name distinct failure modes; *DegradedWithReceipt* names the constructive case where an open finding is admissible without being closed.
4. **A receipt durability invariant** (§4): a receipt must outlive every obligation it covers, and must persist outside the lifecycle, authority domain, and failure domain of the audited object.

Worked Kubernetes cases (§5) exercise the vocabulary across drain-and-reschedule, restore-from-backup, and Node-Problem-Detector-driven cordon. Relation to adjacent results in the series, generalization beyond Kubernetes, limitations, and conclusion follow (§6).

The argument is *constructive*, not merely critical. P25 [1] established that substitution is structurally forced when the target lies outside the observability subspace; P26 [2] formalized the temporal seam under which legitimate binding and consequence viability fail to overlap. Each result is a structural negative. P27 supplies the constructive endpoint: given the prior results, what does it take to govern transitions such that the controller does not silently destroy what was sensed? The answer is not better convergence proofs. It is a separate property — transition admissibility — that the existing correctness vocabulary does not state and cannot enforce.

2. Related Work

Six adjacent literatures recognize components of this failure mode under their own vocabularies. None states the unifying structure as a safety property complementary to convergence.

Neighborhood	Closest claim	What’s missing
Controller verification [4]	$P(\text{eventually current} = \text{desired})$ – liveness under convergence	Dual safety property over the transition that produces convergence
Refinement / abstraction soundness [5, 6]	Refinement preserves the specification	Obligation-soundness as an application-layer constraint, not specification refinement
Failure masking [7]	Redundancy hides faults from clients	<i>Masked-by-recovery</i> as a transition audit state with operator-side cost
Causal tracing / provenance [8, 9, 10]	Reconstruct evidence chains across distributed events	Retention horizons indexed by obligation class; durability invariant for receipts
Forensic readiness [11]	Preserve evidence before incidents occur	Integration with control-loop transition admissibility, not just incident preparation
Kubernetes NPD + chaos engineering [12, 13]	Surface substrate problems; inject faults	No admissibility framework over reconciliation transitions; no formal account of obligation accounting

Anvil-style controller verification [4] proves $P(\text{eventually converged})$ for production reconciliation controllers. It does not, and is not intended to, prove anything about whether the transitions producing that convergence preserve operator obligations. The two properties are orthogonal: a controller can converge correctly and still produce obligation-unsound quotients, and a controller can be transition-admissible while failing to converge.

Refinement-based formal methods [5, 6] establish that a concrete implementation refines an abstract specification when every concrete trace projects to a permitted abstract trace. Obligation-unsoundness is not refinement-unsoundness: the concrete trace projects correctly into the abstract spec; the issue is that *neither* the abstract spec nor its refinement records the obligations that the transition affects.

Failure masking [7] is the closest classical control-systems cousin. Schlichting and Schneider’s fail-stop model deliberately hides certain failures from clients to preserve the clients’ simpler operating semantics. The structural move in P27 is the same — hide substrate state from K -side consumers — but the cost is named differently. In fail-stop, masking is intended; in obligation-unsound reconciliation, masking is incidental, and the operator who needs the masked information is not a downstream client of the controller but an outside party whose obligation the controller did not consult.

Causal-tracing systems [8, 9, 10] reconstruct evidence chains across distributed transitions; PROV-DM [10] formalizes the lineage graph. None of these systems gates a transition’s *admissibility* on whether the transition preserves the relevant causal binding. They are observational instruments, not refusal mechanisms.

Forensic readiness [11] anticipates that incidents will occur and prescribes evidence preservation in advance. Its scope is incident response; P27 extends the discipline into ordinary reconciliation, where the relevant “incident” is the routine transition itself, and the relevant “evidence” is the substrate accusation the transition silently degrades.

NPD [12] surfaces substrate problems as control-plane conditions but does not enforce that reconciliation transitions preserve the substrate accusation across the actions they take in response. Chaos-engineering tooling [13] injects faults but does not audit the obligation accounting of the responses.

The unifying claim: these vocabularies each describe a component of the same failure shape — a control-plane action that satisfies its local correctness condition while degrading operator-side obligations the local condition did not record. P27 identifies the structure and proposes transition admissibility as the missing safety property.

3. Model

3.1 Substrate, control plane, projection

Let B denote the *substrate*: the underlying physical and software state of a system — hardware health, kernel logs, scheduling history, node-level metrics, transient process state, the full causal record of what has happened. Let K denote the *control-plane visible state*: the set of objects, statuses, conditions, labels, and counts that the reconciliation controller can observe and update.

The control-plane visibility is mediated by a projection

$$\alpha : B \rightarrow K$$

specializing the projection construction of *Epistemic Border Control* (Paper 25 [1]). The projection is generally lossy: distinct substrate states $b_1, b_2 \in B$ may share the same control-plane image, $\alpha(b_1) = \alpha(b_2)$. Two such states are *quotient-equivalent* under α .

A reconciliation transition τ is a function $\tau : K \times K_{\text{desired}} \rightarrow (\text{action}, K')$. The controller observes $\alpha(b) \in K$, compares against K_{desired} , selects an action, and the action produces a new substrate state b' with $\alpha(b') = K'$.

3.2 Obligation-unsound quotients

An *obligation* ω is a property of the substrate that some operator relies on for diagnosis, governance, accountability, or further action. We consider the obligation classes appearing in the Lean skeleton accompanying this paper:

- *intent* — the declared goal under which an action was authorized
- *custody* — the chain of accountable handoffs
- *evidence* — the artifacts on which a future testimony could rely
- *causality* — the binding between effects and their substrate-side causes
- *authority* — the standing under which a transition was permitted
- *continuity* — preservation of relevant state across transitions
- *substrate* — accountability of substrate-side accusations

An obligation ω is *relevant* to a transition τ if executing τ may affect ω . We write $\Omega(\tau)$ for the set of relevant obligations.

A reconciliation transition τ produces an *obligation-unsound quotient* with respect to $\omega \in \Omega(\tau)$ when:

1. α conflates substrate states b_1, b_2 that differ along ω — that is, $\alpha(b_1) = \alpha(b_2)$ but $\omega(b_1) \neq \omega(b_2)$ — and
2. τ 's action, which operates uniformly on the shared image $\alpha(b_1) = \alpha(b_2)$ (the controller has no K -level signal that would let it act differently), has obligation-relevant consequences in B that differ across b_1 and b_2 , or erases distinctions across b_1, b_2 that the operator depended on, but
3. τ records no artifact that preserves the distinction across the transition.

The controller does not “distinguish” the substrate states; it cannot, by construction. The unsoundness is that a uniform K -side action produces non-uniform substrate-side consequences along an obligation the projection did not record. In Kubernetes terms: the control-plane controller sees one thing where the operator needs to see two; the transition succeeds on its visible terms while the obligation-relevant consequences in B diverge silently.

3.3 Transition admissibility

Per-obligation outcomes are drawn from a small algebra. Each transition produces, for each affected obligation, one of:

- **preserved** — the obligation survives the transition unchanged
- **transferred** — custody passes to another accountable party with a recorded handoff
- **discharged** — the obligation is satisfied and closed under explicit acknowledgment
- **degradedWithReceipt** — the obligation is partially impaired, but the impairment is recorded with a durable receipt
- **openFinding** — the obligation is not closed; the open status is durably recorded

These outcomes are wrapped in an `Accounting` layer:

$$\text{Accounting} ::= \text{unaccounted} \mid \text{accounted}(\text{Outcome})$$

A transition is *transition-admissible* under obligations $\Omega(\tau)$ and a receipt r when:

$$\text{admissible}(\Omega, r) \iff (\forall \omega \in \Omega. r.\text{account}(\omega) \neq \text{unaccounted}) \wedge (\forall \omega \in \Omega. r.\text{durability} \geq H(\omega))$$

where $H(\omega)$ is the retention horizon required by obligation ω .

Two clauses are doing different work. The *no-unaccounted* clause requires the transition to record an outcome for every obligation it affects; silent obligation degradation is the inadmissibility condition. The *durability* clause requires the receipt to outlive the obligation horizon; a receipt that ages out before the obligation it covers reduces to an unaccounted outcome retroactively.

The Governor primitive is preserved by this algebra. An `openFinding` is admissible — refusal to launder uncertainty into closure is honesty with a pager, not a failure mode. The inadmissible case is `unaccounted`: a transition that affects an obligation and produces *no* recorded outcome for it. The corresponding theorem in the accompanying Lean skeleton, `unaccounted_implies_inadmissible`, is proven; the dual `open_finding_admissible_with_durability` is also proven, establishing that the model does not collapse `openFinding` into refusal.

Transition admissibility is *orthogonal* to convergence. A transition can converge (current = desired in K) while failing admissibility (some $\omega \in \Omega(\tau)$ is unaccounted). A transition can be admissible while failing to converge (every obligation is accounted, but the action did not achieve the desired K -state). The two properties index different objects: convergence indexes the control plane’s terminal state; transition admissibility indexes the per-transition record of obligation accounting.

4. Audit States and Receipt Durability

4.1 Audit states

The audit-state vocabulary names the operator-side classification of a transition. We distinguish nine states; three are the operationally novel contribution.

- **Observed** — the transition occurred; outcome under analysis
- **Confirmed** — the transition is admissible; all obligations accounted; receipts durable
- **Superseded** — a later transition correctly subsumes this one’s obligations
- **Masked** — the transition appears converged in K but degraded an obligation in B without recording it
- **Orphaned** — an obligation’s causal binding was destroyed by the transition, leaving the substrate accusation un-attributable
- **DegradedWithReceipt** — the transition impaired an obligation but recorded the impairment as an openFinding with a durable receipt
- **Resolved** — an open finding is independently closed by the substrate custodian
- **Invalidated** — a prior finding is withdrawn under a corrected substrate observation
- **CannotTestify** — the auditor lacks standing or evidence to classify; the structured non-answer

Masked, *Orphaned*, and *DegradedWithReceipt* are the operationally novel states. The first two name distinct failure modes; the third names the constructive case where an open finding is admissible without being closed.

Masked corresponds to obligation-unsound quotienting where the operator-side cost is the loss of evidence or substrate accountability. The classical case: a transition restores convergence by destroying the artifacts on which substrate-side testimony would have relied. *Orphaned* corresponds to the loss of *causal binding*: the substrate event and the artifact state both existed at the time of the transition, but the controller did not bind them, and one or both has since aged out of retention. The substrate is no longer attributable to the artifact’s history.

DegradedWithReceipt is the audit state that preserves the Governor primitive. A transition that impairs an obligation but records the impairment as an open finding with a receipt durable beyond the obligation’s horizon is admissible. The system reports its state honestly: service available, finding still open. This is the audit state under which a reconciliation controller can act in degraded conditions without quotienting away the substrate accusation. The pattern is not merely allowed; it is the architectural shape required by transition admissibility when convergence and obligation preservation cannot both be achieved by the same transition.

CannotTestify matches the structured-non-answer move in the Governor’s verdict algebra: an auditor that lacks standing or evidence to classify a transition emits *CannotTestify* rather than a confident *Confirmed* or a confident *Masked*. This refuses the temptation to launder ignorance into either category.

4.2 Receipt durability invariant

A *receipt* records the obligation accounting for a transition. Receipts have a *durability horizon* — the length of time the receipt is required to remain accessible, attributable, and unmodified.

The durability invariant: for any receipt r covering obligations $\Omega(\tau)$,

$$H(r) \geq \max_{\omega \in \Omega(\tau)} H(\omega).$$

Receipt durability is *relative*, not absolute — $H(r)$ is bound to the longest-lived affected obligation, not to a fixed wall-clock duration. A receipt is durable for its purpose iff it outlives every $\omega \in \Omega(\tau)$.

The horizon $H(\omega)$ is obligation-specific. Evidence-class obligations may carry short horizons in some regimes (debug logs, ephemeral traces) and long horizons in others (audit trails subject to regulatory retention). Causality-class obligations typically carry horizons matched to the slowest investigation timeline relevant to the system. Substrate-accountability obligations carry horizons matched to the substrate locus’s remediation cycle, which may span hardware-replacement intervals.

A second invariant follows from the first. Receipts must persist *outside* the lifecycle, authority domain, and failure domain of the audited object. A receipt stored in the pod whose drain it records is not durable past the pod’s lifecycle. A receipt stored in the node whose cordon it records is not durable past the node’s replacement. A receipt under the same authority as the action it records can be silently revoked by that authority. The architectural consequence is that admissible receipts must live in an artifact whose lifecycle is independent of the transition they cover — typically a control-plane object with its own retention policy (an event record, a status condition, a custody-bound log entry), or an external system whose retention horizon exceeds the obligation horizon.

We do not formalize full self-governance of the audit plane in this draft. Recursive receipt admissibility — what auditor governs the auditor — is sibling work, deferred (§6).

5. Worked Cases

Three Kubernetes scenarios exercise the vocabulary. Each is presented at narrative level; the structural per-case fields (substrate state, projection gap, destroyed obligations, predicate forced, receipt requirement) appear in the supplementary `WORKED_CASES.md`. The first two name failure modes; the third names the constructive case.

5.1 Masked recovery: drained pod, destroyed evidence

A pod fails on a node whose disk hardware has been degrading. The kernel disk-error log entries are accumulating in B but have not been promoted to a control-plane node condition. From the kubelet’s perspective, the container exited; that information reaches K . The reconciliation controller drains the pod — destroying the container’s stdout/stderr and any kernel messages tied to the pod’s lifecycle — and reschedules onto another node selected by the existing scheduling policy. In the absence of a node-condition signal, that node may be in the same node class with the same substrate risk profile.

Convergence is satisfied at $t + \epsilon$: pod healthy, replicas correct, service Ready. The transition’s K -level audit reports success. But the projection gap was load-bearing: the substrate-disk evidence existed in B at $t - \epsilon$ and is gone at $t + \epsilon$; the link between the pod failure and the substrate state is unbound; the failing node

remains in the schedulable pool with no recorded accusation. The next pod scheduled there inherits the same risk silently.

This is the canonical *Masked* state. The predicates forced are `substrateAccusation` (the substrate carries a standing finding that should outlive the transition) and `maskedByRecovery` (K reports recovery while the substrate accusation remains open and undeclared). The receipt required to restore admissibility is one that references the suspect-node identity, the destroyed-evidence class, and the substrate accusation as an open finding — and persists outside both the pod and node lifecycles affected by the transition.

5.2 Orphaned causality: restored volume, aged-out cause

A volume is detected as corrupted. The reconciliation policy restores the affected replica from backup, potentially onto a different node. Integrity checks confirm the restored data; convergence advances.

The corruption is traceable, in principle, to a specific substrate event at some earlier time t_{event} — a hardware fault, a neighbor-pod resource interference episode, a scheduling-induced disk contention, or a related disturbance. The substrate-layer evidence (scheduling history, hardware logs, neighbor-pod activity, node metrics around t_{event}) exists at the time of corruption detection but is subject to retention horizons typically shorter than the corruption-detection-to-investigation horizon. By the time someone asks *what caused the corruption*, the substrate evidence has aged out.

This is the *Orphaned* state. The data is back; the causal chain to the substrate event is not. The relevant failure is not that the link was wrong but that the link was never durably bound at the moment when evidence on both sides was live. The predicates forced are `causalBinding` (the binding between artifact state and substrate accusation must be established while both are live) and `receiptOutlives` (receipt durability must exceed the substrate-event retention horizon, not merely the artifact-recovery horizon). The receipt required is one bound at the time of corruption detection to the substrate evidence then available, with durability exceeding the substrate-event retention horizon. The restore receipt alone is insufficient; what is needed is a binding receipt that captures the causal-chain endpoints while both still exist.

5.3 DegradedWithReceipt: thermal-event drain, durable accusation

A node is experiencing a thermal event — sustained thermal load or cooling failure — with durable substrate-side evidence (thermal sensor data, possibly hardware-vendor diagnostics). The Node Problem Detector [12] writes a node condition recording the accusation. The reconciliation controller cordons the node, drains pods to other nodes, and marks the substrate accusation as an open finding bound to the node identifier.

Convergence is satisfied: pods relocated, replicas correct, downstream service available. Transition admissibility is *also* satisfied: every affected obligation is accounted for (the thermal-event evidence is preserved by NPD; the substrate accusation is bound to the node identifier; remediation responsibility is attributable to the substrate custodian), and the receipt durability matches the longest-affected obligation. The audit state is *DegradedWithReceipt*: downstream service operates in a degraded mode because the substrate accusation remains open, and the open status is durable because the receipt outlives the pod and node lifecycles affected by the transition.

The predicates exercised are `receiptOutlives` (receipt horizon $\geq$ obligation horizon), `substrateAccusation` (named and durably recorded), and `openFinding` (an admissible Outcome class already defined in the Lean skeleton). The Governor primitive is operationally satisfied: permit action, deny closure, leave finding open.

This case is structurally distinct from Cases 5.1 and 5.2. Both of those record a transition that converged in K while quotienting an operator-side obligation. Case 5.3 records a transition that converged in K and preserved the operator-side obligation through a durable receipt outside the audited lifecycle. The pattern generalizes: when convergence and full obligation preservation cannot both be achieved by the same transition, transition admissibility prescribes recording the partial obligation as an open finding with a durable receipt, not quotienting it.

6. Relation to Series

P25 [1] established that under observability asymmetry — when the target variable is not directly sensed and the available observables load on a proxy — a controller is structurally forced to substitute the proxy for the nominal target. The result is *spatial*: the controller regulates the wrong variable because it cannot see the right one. P26 [2] established a *temporal* sibling: when the time-axis evidence required for legitimate binding does not arrive before the consequence window closes, the binding event is structurally premature, belated, or impossible.

P27 occupies the position the two prior results leave open. Given that the controller may face an observability asymmetry (P25) and a temporal seam (P26), what does it take for the transitions the controller executes to *preserve* the conditions under which a different observer could later testify? The answer is transition admissibility: a property of the per-transition record, not of the controller’s terminal state.

Retention horizons follow the two-curve framing of Paper 26 [2]. The consequence-viability curve $c(t)$ of P26 specializes per obligation class — evidence, causality, substrate-accountability — and the receipt durability invariant of §4.2 requires the receipt horizon to outlive the longest-lived affected obligation. P27 borrows the horizon machinery from P26 without absorbing the full premature/belated duality; the duality remains a P26 result.

P23 [3] established hidden compensation: an operational controller’s identity-continuity can fail through latent human compensation invisible to the control-plane audit. Hidden compensation is one downstream consequence of obligation-unsoundness: when the substrate accusation is destroyed by reconciliation, the operator who carries the unrecorded causal continuity is the hidden compensator. The relation is cited, not central.

7. Generalization

Kubernetes is the most-tested specimen for reconciliation-loop pathologies and the most widely operated at scale. The structural model — controller observing a lossy projection of substrate state, executing transitions that may affect obligations the projection does not record — generalizes to the broader class of reconciliation controllers: cloud autoscalers, rollback controllers, agent retry frameworks, CI/CD repair pipelines, service-mesh retry policies, database failover orchestrators, configuration management daemons. The audit-state vocabulary applies wherever a transition can converge in the controller’s local view while degrading an obligation the controller does not consult.

The model does not require that the substrate be physical hardware. Any state that the controller’s projection collapses — application-layer state, business-layer obligations, regulatory-class artifacts — can carry obligations the controller’s specification did not record. The Kubernetes case is the cleanest instance because the projection $\alpha : B \rightarrow K$ is explicit and the controller’s specification is mechanized; the broader claim is that the same shape recurs wherever a reconciliation controller operates over a lossy projection and a non-trivial obligation set.

8. Limitations

Four limitations require explicit naming.

The Lean skeleton carries two True-placeholder theorems. The companion Lean file at `~/git/lean/LeanProofs/Adm` proves three substantive theorems against the local admissible definition (`unaccounted_implies_inadmissible`, `short_receipt_horizon_inadmissible`, `open_finding_admissible_with_durability`). Two further theorems — `masked_recovery_not_resolved` and `orphaned_causality_inadmissible` — are stated as True and discharged trivially, with deferred-real-statement docstrings. Promoting them to load-bearing theorems requires sibling vocabulary not yet declared: a substrate-accusation predicate and a K -transition-outcome predicate for the former, a causal-binding predicate and an attributability predicate for the latter. Each required predicate is itself a kernel commitment, and the discipline applied here is to upgrade a placeholder theorem only when the underlying vocabulary is independently justified by a downstream proof obligation — never by introducing kernel predicates solely to discharge a placeholder. The placeholders are limitations of this draft, not failures of the model.

Two predicates surfaced by the worked cases cleared the two-case forcing gate but are not promoted. The supplementary `WORKED_CASES.md` records, under its own discipline rule, that `substrateAccusation` (forced by Cases 5.1 and 5.3) and `receiptOutlives` (forced by Cases 5.2 and 5.3) have appeared as predicates-forced in at least two independent cases. By the supplementary file’s gate, both are eligible for Lean kernel promotion. They are *not* promoted in this draft: a kernel predicate without a downstream theorem that composes with the existing `AuthorizedStep / Corrective` layer would be constitution-by-typechecker, which the kernel discipline explicitly rejects. The two cleared predicates are recorded here as deferred-but-eligible. Future work — either a downstream theorem in a sibling kernel module or a P27 follow-up establishing the composition — would justify promotion.

Receipt-recursion is scope-limited. §4.2 states the receipt durability invariant and the persistence-outside-lifecycle requirement. It does not formalize the self-governance question — *what auditor governs the auditor* — beyond noting that admissible receipts must persist outside the authority domain of the action they record. Recursive admissibility (the auditor’s own transitions are themselves subject to transition admissibility) is a real question that the model permits in principle but does not develop. Treating the audit plane’s own transitions under the same admissibility predicate is sibling work; the present draft establishes the predicate and the worked-case vocabulary without recursing.

Contested or partially-operative obligations are treated as phase-sensitive. The audit-state vocabulary handles cases where an obligation’s status changes over time — open at t_1 , resolved at t_2 , invalidated at t_3 — through the *Resolved* and *Invalidated* states. Cases where an obligation is *in motion but not complete* (a proposed retention policy, a draft governance rule, a contested standing claim) are not given their own classification; they appear in the audit log as a sequence of state transitions through *Observed*, *Degraded*, *WithReceipt*, and either *Resolved* or *Invalidated* as the underlying obligation status crystallizes. Whether such cases warrant their own classification is held open; the issue is recorded as a limitation pending a forcing case where phase-sensitive treatment is insufficient.

9. Conclusion

A reconciliation controller can be correct on its declared correctness property — eventual convergence to a desired state — while remaining unsound on a property the declared specification does not name. The unstated property is whether the transitions producing convergence preserve the obligations on which a different observer might later rely. We have given that property a name (transition admissibility), an algebra (obligation outcomes under an Accounting wrapper), an invariant (receipt durability), a vocabulary

for the failure modes and the constructive case (Masked, Orphaned, DegradedWithReceipt), and three Kubernetes specimens that exercise the distinctions.

The result is small but specific. It does not replace convergence as a controller specification; it complements convergence with a per-transition record requirement. It does not assert that obligation-unsound quotients are everywhere; it identifies the structural conditions under which they arise and the architectural response that prevents them — receipts that outlive their obligations, recorded in artifacts that outlive the audited lifecycles.

P25’s spatial substitution and P26’s temporal seam describe what a controller cannot achieve when its sensors and its time-axis evidence fail to cover what it is supposed to govern. P27 describes what the controller can be required to *not destroy* when it acts under those constraints. Convergence specifies the terminus; transition admissibility specifies the trail. Both are required for a controller to remain accountable to the operator who depends on it.

References

- [1] Beck, J. (2026). Epistemic Border Control as Proxy Regulation Under Partial Observability. *At Framework series*, Paper 25. Zenodo. doi:10.5281/zenodo.20145155
- [2] Beck, J. (2026). Premature Commitment, Belated Consequence, and the Empty Binding Window. *At Framework series*, Paper 26. Zenodo. doi:10.5281/zenodo.20189777
- [3] Beck, J. (2026). Ops Is Control with a Non-Self-Identical Controller. *At Framework series*, Paper 23. Zenodo. doi:10.5281/zenodo.19715301
- [4] Sun, X., Ma, W., Gu, J. T., Ma, Z., Chajed, T., Howell, J., Lattuada, A., Padon, O., Suresh, L., Szekeres, A., & Xu, T. (2024). Anvil: Verifying Liveness of Cluster Management Controllers. *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. arXiv:2404.06777.
- [5] Abadi, M., & Lamport, L. (1991). The Existence of Refinement Mappings, *Theoretical Computer Science*, 82(2), 253–284. doi:10.1016/0304-3975(91)90224-P
- [6] Lamport, L. (1994). The Temporal Logic of Actions, *ACM Transactions on Programming Languages and Systems*, 16(3), 872–923. doi:10.1145/177492.177726
- [7] Schlichting, R. D., & Schneider, F. B. (1983). Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems. *ACM Transactions on Computer Systems*, 1(3), 222–238.
- [8] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report dapper-2010-1.
- [9] Barham, P., Donnelly, A., Isaacs, R., & Mortier, R. (2004). Using Magpie for Request Extraction and Workload Modelling. *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04)*, 259–272.
- [10] Moreau, L., Missier, P., Belhajjame, K., B’Far, R., Cheney, J., Coppens, S., Cresswell, S., Gil, Y., Groth, P., Klyne, G., Lebo, T., McCusker, J., Miles, S., Myers, J., Sahoo, S., & Tilmes, C. (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.

- [11] Rowlingson, R. (2004). A Ten Step Process for Forensic Readiness. *International Journal of Digital Evidence*, 2(3), 1–28.
- [12] Kubernetes contributors. Node Problem Detector. Project documentation, [kubernetes/node-problem-detector](#).
- [13] Cloud Native Computing Foundation. Litmus and Chaos Mesh project documentation. CNCF Chaos Engineering Working Group materials.

Status: v0.2/1.0-candidate, 2026-05-14. Companion Lean skeleton: `~/git/lean/LeanProofs/Admissibility.lean` (three real proofs against the local admissible definition; two `True-placeholder` theorems with deferred-real-statement docstrings). Worked-case scaffolding: `preprint/27-obligation-unsound-reconciliation/WORKED_CASES.md`. Not yet pushed to Zenodo.